LABORATORY 4 - **JQUERY LIBRARY**

4.1 Introduction

JavaScript language is, in fact, a version of ECMAScript, and present some implementation differences between browsers. Although sustained efforts are carried on with the purpose of reducing these differences, JavaScript code created for production still needs to be tested extensively on the most common browsers to ensure the same user experience.

As a response to these issues and on an attempt to reduce the testing effort, multiple cross-browser JavaScript libraries have been implemented. They ensure code compatibility and functionality consistence on all targeted browsers, providing also a higher level API for rich applications development. One of the most popular libraries today is jQuery.

4.1.1 Laboratory objectives

Present basic aspects and functionalities of the jQuery library. Focus on HTML tags interaction and DOM manipulation.

4.2 Introduction to jQuery

It is important to understand that jQuery is not a programming language, but a library of functions that is implemented in JavaScript and ensures cross-browser compatibility. In other words, it hides the differences in JavaScript code execution and provides to the users an identical behavior of the web application independent on the browser used to access it. This is achieved through the identification of available native APIs and the call of browser-specific functions whenever deemed necessary.

4.2.1 Import jQuery library

In order to access the jQuery function from your own JavaScript code, it is first necessary to include it in the HTML file. The statement is identical with the inclusion of any other external JavaScript file. As it is written in JavaScript, jQuery is actually included directly as source-code and compiled at runtime by the browser at each page load. The last available version of the library can be downloaded from: http://jquery.com/download/

There are two main approaches on including jQuery library in the HTML file:

(1) download the source code on the application's web server, next to other JS files that you have, and include it in HTML as follows:

```
<script type="text/javascript" src="js/jquery.min.js"></script>
```

(2) use a Content Delivery Network (CDN) to optimize delivery time and save bandwidth, by loading the source code directly from the source

```
<script type="text/javascript"
        src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js">
</script>
```

4.2.2 Use jQuery functions

It is very important to note that jQuery functions can be invoked only after the library has been downloaded by the browser, parsed and initialized. For this matter, any JavaScript code that relies on jQuery should run only after the page is fully loaded and DOM elements become available.

It is thus recommended to make use of the **window.onload** event for starting your own code execution only after all JavaScript resources have been downloaded, parsed and initialized. For these cases, jQuery provides the following syntax:

```
$( document ).ready( function() {
   // code here will be executed when all DOM and JavaScript resources are available
});
```

After jQuery library has been initialized, its functions can be accessed through two different syntactical constructions that have identical results: **jQuery** and **$**. If there are no conflicts (ie. JavaScript code that redefines the **$** function), the exact same functions are called through any of the two mentioned constructions. For example, the code above can be rewritten as:

```
jQuery( document ).ready( function() {
   // code here will be executed when all DOM and JavaScript resources are available
});
```

Using any of the **jQuery** or **$** symbols in any other JavaScript files (or directly in HTML, in <script> sections) is accepted by the browser only if the reference to the jQuery library precedes these occurrences in HTML. More details can be found at: http://learn.jquery.com/using-jquery-core/document-ready/ .

4.2.3 Manipulate HTML tags using jQuery

Identical with plain JavaScript code, the first step in interacting with HTML tags is getting a reference to the desired element's representation in the DOM. In most cases, jQuery provides to developers a broader approach than native JavaScript in searching for HTML tags, based on any valid CSS selector.

However, in all situations where jQuery selectors do not have a direct correspondence in native JavaScript calls, the search of the elements is implemented through successive DOM queries which can get time and resources consuming in the case of complex HTML documents.

In the following examples, the use of $ sign for the variables names is just a visual queue to indicate that the returned references are actually jQuery objects (more details in the following section) and not native DOM elements. Selectors examples:

```
// select element with a specific ID
var $element = $( "#myId" );  // Note: IDs must be unique per HTML document.

// select all elements that on execution time have a specific class attached
var $elements = $( ".myClass" );

// select all elements of a specific HTML type
var $elements = $( "input" );

// select all elements of a specific HTML type and attribute value
var $elements = $( "input[name='first_name']" );

// select all LI elements of the unordered lists with class "people"
// that have in parents list the element with ID "contents"
var $elements = $( "#contents ul.people li" );
```

More examples of available selectors, including jQuery specific ones, are available at: http://learn.jquery.com/using-jquery-core/selecting-elements/ .

4.2.4 jQuery objects

The result of each of the examples above is a jQuery object that encapsulate all the DOM elements that match the selector used. Even if there is no match found, the result is not **null** but a jQuery object that has no references encapsulated. Consequently, in order to verify if a selector has returned any elements we cannot use:

```
if ( $elements == null ) { }
```

as this expression will always be **false** (the selectors are not returning **null**). The correct approach is to verify the **length** of the returned jQuery object:

```
if ( $elements.length > 0 ) { }
```

There are situations in which we need a direct reference to the DOM element and not the jQuery encapsulation. For these cases the function **[jQueryObject].get(index)** defined in jQuery can be used:

```
var jQuery_element = $( "#myElement" );
var DOM_element = document.getElementById( "myElement" );

// this will print "true"
console.log( DOM_element == jQuery_element.get( 0 ) );
```

We can thus conclude that jQuery objects are maintaining a reference to the DOM elements that have been matched by a specific selector. On these encapsulations we can apply all the functions defined in the jQuery library (more details at: http://learn.jquery.com/using-jquery-core/jquery-object/).

4.2.5 Manipulate HTML elements

One of the important things to remember when working with jQuery library is that a jQuery object hold the references to **zero**, **one** or **several** DOM elements. All the functions defined in jQuery that are called on the object will be applied to all the referenced DOM entities. For example, in order to add class “customized” to all the paragraphs from an HTML document, we will simply use the following line:

```
$( "p" ).addClass( "customized" );
```

If used correctly, this can be very powerful and will allow complex implementations with few lines of code. However, if used incorrectly, this can have undesired side-effects:

```
$( "p" ).html( "Content that is meant for a single paragraph" );
```

will **change the content of each paragraph from the page** to “Content that is meant for a single paragraph”.

In order to complete the exercises proposed for this laboratory work you will need the following functions from the jQuery library:

.html()	https://api.jquery.com/html/

returns the content of the DOM element as HTML

.html(htmlString) https://api.jquery.com/html/

sets the content of the DOM element to **htmlString**

- ! this function will parse the htmlString and will automatically add to the DOM any tag found
- ! **this function is vulnerable to JavaScript code injection** (through <script> elements)

.text() https://api.jquery.com/text/

returns the content of the DOM element as plain text

.text (plainText) https://api.jquery.com/text/

sets the content of the DOM element to **plainText**

- ! if plainText contains valid HTML code it will be ignored and no DOM elements are created - the text will be displayed exactly as provided

.val() https://api.jquery.com/val/

returns the content of an INPUT element

.val(value) https://api.jquery.com/val/

sets the content of an INPUT element

.attr(attrName) http://api.jquery.com/attr/

returns the value of the **attrName** attribute of the element

.attr(attrName, attrVal) http://api.jquery.com/attr/

sets the value of the **attrName** attribute of the element

.addClass (className/s) https://api.jquery.com/addClass/

applies to the element the CSS class (or CSS classes) from **className/s** (the parameter can be a list of classes separated through spaces)

.removeClass (className/s) https://api.jquery.com/removeClass/

removes from the list of applied CSS classes the class (or classes) specified in **className/s** (the parameter can be a list of classes separated through spaces)

! if this function is called without parameters it will remove all the CSS classes applied on the element

.hide() https://api.jquery.com/hide/

instantaneous hiding of a visible HTML element by setting **width** and **height** properties to 0px.

i. this function has multiple signatures enabling also an animated transition between visible and invisible states

.show() https://api.jquery.com/show/

instantaneous display of an HTML element previously hidden using *.hide()*, by setting **width** and **height** properties to their initial values

i. this function has multiple signatures enabling also an animated transition between invisible and visible states

.on(eventString, handlerFunc) https://api.jquery.com/on/

allows registration of **handlerFunc** to be called when the **eventString** event is raised; the handler function receives the **event** parameter.

.animate(prop, delta, onCompletion) https://api.jquery.com/animate/

animates a visible DOM element by continuously updating the properties mentioned in **prop** over the **delta** time interval, until the desired values are set

- **prop** – JavaScript object that specifies the visual properties that should be animated and the desired final values; for example, in order to animate the size of an object (width and height) from *current_values* to *250 px 300 px*:

```
prop  = {
    height: 300px,
    width: 250px
}
```

- **delta** – duration of animation in milliseconds
- **onCompletion** – function called when the animation is complete

.delay (delta)	https://api.jquery.com/delay/ allows the delay in execution launching of functions applied to the same HTML element in the same processing queue, for **delta** milliseconds • this is different in outcome compared to the native JavaScript function setTimeout()
.queue (queuedFunction)	https://api.jquery.com/queue/ add a new function to the current processing queue of an HTML element ! when queuedFunction has finished execution, .dequeue() **must be invoked** or the processing queue will stall
.dequeue()	https://api.jquery.com/dequeue/ launches the following function in the execution queue

4.2.6 DOM traversal

When we have a reference to a DOM element, encapsulated in a jQuery object, we can use it as a starting point for a DOM traversal in one of the following three directions: (more details and the complete list of available functions can be found on: http://learn.jquery.com/using-jquery-core/traversing/):

(1) towards the root of the DOM (searching through the ancestors of the current element), using one of the following functions:
- **.parent([selector])** – https://api.jquery.com/parent/
  returns the parent element if it matches the **selector**
- **.parents([selector])** – https://api.jquery.com/parents/
  returns all the ancestor nodes (between the current element and the root of the DOM), that match the **selector**.

(2) on the same level (HTLM elements having the same parent element), using one of the following functions:
- **.prev([selector])** – https://api.jquery.com/prev/
  returns the previous sibling, if it matches the **selector**
- **.prevAll([selector])** – https://api.jquery.com/prevall/
  returns all the previous siblings that are a match for the **selector**
- **.next([selector])** – https://api.jquery.com/next/
  returns the next sibling, if it matches the **selector**
- **.nextAll([selector])** – https://api.jquery.com/nextAll/
  returns all the next siblings that are a match for the **selector**
- **.siblings([selector])** – https://api.jquery.com/siblings/
  returns all the HTML elements that have the same parent (siblings) and are matching the **selector**

(3) towards the leaves of the DOM (searching through the child nodes), using one of the following functions:
   - **.children([selector])** – https://api.jquery.com/children/
     returns all the HTML elements that are direct children of the current element and match the **selector**
   - **.find([selector])** – https://api.jquery.com/find/
     searches the DOM through all the children of the current node and returns the elements that match the **selector**

4.2.7 DOM manipulation

In the Web 2.0 interactive applications it is often necessary to create, add, move or remove HTML elements on the fly, as a response to user's actions or according to data display requirements. jQuery provides a set of functions for easy HTML nodes CRUD operations:

$el.insertAfter($target)	http://api.jquery.com/insertAfter/ insert all the HTML elements from **$el** after each element from **$target**
$el.after($content)	http://api.jquery.com/after/ insert all the HTML elements from **$content** after each element from **$el**
$el.insertBefore($target)	http://api.jquery.com/insertBefore/ insert all the HTML elements from **$el** before each element from **$target**
$el.before($content)	http://api.jquery.com/before/ insert all the HTML elements from **$content** before each element from **$el**
$el.appendTo($target)	http://api.jquery.com/appendTo/ insert all the HTML elements from **$el** as the last child elements of each DOM element from **$target**
$el.append($content)	http://api.jquery.com/append/ insert all the HTML elements from **$content** as the last child elements of each DOM element from **$el**
$el.prependTo()	http://api.jquery.com/prependTo/

	insert all the HTML elements from **$el** as the first child elements of each DOM element from **$target**
$el.prepend()	http://api.jquery.com/prepend/ insert all the HTML elements from **$content** as the first child elements of each DOM element from **$el**

4.2.8 Chaining function calls

One of the most useful particularities of the jQuery library is that almost all the functions return a reference to the jQuery object they have been invoked on. This enables **function calls chaining**, which allows developers to do a lot with very little code and to optimize resources (single DOM search for multiple updates and no persistent references). For example, the following changes:

- remove class ***warning***
- add class ***success***
- update displayed message
- make sure the HTML element is visible

can be expressed through:

```
$("#myElement").removeClass("warning").addClass("success")
               .text("Operation completed successfully").show();
```